VEHICLE SERVICE TRACKER

Micro Project Report

Submitted for the Partial Fulfillment of the Requirements

for the Award of the Degree of

Master of Computer Applications

By

Nandana Dinesh – MITS25PMCA042
Muhammad Shafin E A -MITS25PMCA040
Mohammed Iqbal M A-MITS25PMCA039

Under the Guidance of

Dr Geethu S

Assistant Professor

Department of Computer Applications

Varikoli P O, Puthencruz, Ernakulam District, Kerala

(Affiliated to A P J Abdul Kalam Technological University, Thiruvananthapuram, Kerala)

October – 2026

Department of Computer Applications

BONAFIDE CERTIFICATE

*This is to certify that the Micro Project Report entitled "**VEHICLE SERVICE TRACKER**" has been submitted by **Nandana Dinesh, Reg No: MITS25PMCA042** to Muthoot Institute of Technology and Science, Varikoli, for the partial fulfillment of the requirements for the award of the degree of Master of Computer Applications (MCA), and is a bonafide record of the project work carried out by her under our supervision and guidance during the academic year 2026–27.*

The Micro Project was carried out as part of the Third Semester MCA course A250201/CA330U- MOBILE APPLICATION DEVELOPMENT LAB.

Place: Varikoli

Date:

Project Guide
Dr Geethu S

HoD
Dr Saritha K

Submitted for the Final Evaluation held on ……

Name and Signature of the Examiner

DECLARATION

I, the undersigned, hereby declare that the Micro Project Report “Vehicle Service Tracker”, submitted for the partial fulfilment of the requirements for the award of degree of Master of Computer Applications of the APJ Abdul Kalam Technological University, Kerala is a bonafide work done by me under the supervision of Dr Geethu S. This submission represents my ideas in my own words. I also declare that I have adhered to ethics of academic honesty and integrity and have not misrepresented or fabricated any data or idea or fact or source in my submission. This report has not been previously formed the basis for the award of any degree, diploma or similar title of any other University.

1. Ms. Nandana Dinesh Reg. No: MITS25PMCA042 Signature

Place: Varikoli

Date:

Acknowledgement

I would like to express my sincere gratitude to all those who supported and guided me throughout the development of the Vehicle Service Tracker micro project. This project would not have been possible without their invaluable contributions and assistance.

I am deeply thankful to my mentor, Dr. Geethu S, for the expertise, guidance, and encouragement. I am grateful to my peers and friends who provided valuable insights, engaged in meaningful discussions, and offered their assistance whenever needed. Their diverse perspectives enriched the project's outcome.

I would like to acknowledge the open-source community for developing and maintaining the libraries, frameworks, and tools that I utilized in this project. Their contributions have greatly facilitated the implementation process.

Place: Varikoli
Date:

CONTENTS

CHAPTER	PAGE NO.
ABSTRACT	1
1. INTRODUCTION	2
1.1 Introduction and Background	2
1.2 Problem Statement	2
1.3 Objectives of the Project	2
1.4 Scope of the Project	3
2. SYSTEM ANALYSIS	4
2.1 Existing System	4
2.2 Proposed System	4
3. SYSTEM DESIGN AND IMPLEMENTATION	5
3.1 System Requirements	5
3.1.1 Hardware Requirements	5
3.1.2 Software Requirements	5
3.2 System Architecture	5
3.3 Functional Modules and Workflow	7
3.3.1 Create Module	7
3.3.2 Read Module	7
3.3.3 Update Module	7
3.3.4 Delete Module	7
3.3.5 Local Database Module	8
3.3.6 Application Workflow	8
3.4 Implementation Details	8
3.4.1 Database Design	9
3.4.2 Adding Service Records	9
3.4.3 Viewing Service Records	10
3.4.4 Updating Service Records	10
3.4.5 Deleting Service Records	10
3.4.6 Implementation Limitations	10
4. RESULTS AND DISCUSSION	11
4.1 Application Testing and Results	11
4.1.1 Create Operation	11
4.1.2 Read Operation	11
4.1.3 Update Operation	12
4.1.4 Delete Operation	13
4.1.5 Functional Testing Summary	13
4.2 Results and Discussion	14
5. CONCLUSION AND FUTURE WORK	15

ABSTRACT

Vehicle Service Tracker is an Android-based application developed to help vehicle owners record and manage the service history of their vehicles. Many owners depend on memory, paper bills, or scattered messages to remember when a vehicle was last serviced, what work was done, and how much it cost. This makes it difficult to maintain an accurate service history. The application addresses this problem by providing a simple and organised way to store service details in one place on the mobile device.

The application is developed using Java and Android development tools, with XML layouts for its user interface. Service data is stored locally in an SQLite database on the device. The project is intentionally limited to the four basic data operations, namely Create, Read, Update, and Delete (CRUD), so that it demonstrates core mobile application development concepts such as activities, layouts, user input handling, list display, and local data storage.

Using the application, a user can add a service record containing the vehicle name, registration number, service type, service date, odometer reading, cost, next due date, and notes. All saved records are displayed in a list on the home screen. The user can open a record to view its complete details, edit the stored information, or delete a record after confirming the action. Input validation is applied so that required fields are not left empty and numeric fields accept only valid values.

Because all data is stored on the device, the application works without an internet connection and does not require user accounts or external services. The application does not include reminders, cloud backup, or reports, as these are outside the scope of this micro project.

Functional testing was carried out for each CRUD operation. The tests covered adding a record, viewing the list and the record details, editing a record, deleting a record with confirmation, rejecting invalid input, and retaining saved records after the application was closed and reopened. Overall, Vehicle Service Tracker aims to make vehicle service information easier to record, review, and maintain.

CHAPTER 1

INTRODUCTION

1.1 Introduction and Background

Regular servicing is essential for keeping a vehicle safe, reliable, and efficient. Oil changes, brake inspections, tyre replacements, battery checks, and general servicing must be carried out at suitable intervals. To plan these tasks properly, owners need to know when the last service was done, what was performed, the odometer reading at that time, and the amount spent.

Vehicle Service Tracker is an Android application that allows owners to maintain this information digitally. It provides a simple interface for creating, viewing, updating, and deleting service records. The application uses SQLite for local storage, so the records are kept on the device and are available even when the device is offline.

The application demonstrates the use of Android activities, XML layouts, RecyclerView, input validation, dialogs, and SQLite database operations, which are the fundamental concepts covered in the Mobile Application Development Lab.

1.2 Problem Statement

Vehicle owners often keep service details in paper bills, notebooks, phone messages, or simply in memory. Such records are easy to misplace, hard to search, and difficult to keep up to date. As a result, owners may forget the date of the last service, repeat or delay important maintenance, or be unable to provide a clear service history when selling the vehicle or visiting a workshop.

Therefore, there is a need for a simple mobile application that stores vehicle service details in a structured form and lets the user add, view, correct, and remove records easily. The application should be lightweight, easy to use, and should work without an internet connection.

1.3 Objectives of the Project

The main objective of Vehicle Service Tracker is to provide a simple and reliable way to maintain vehicle service records on an Android device. The specific objectives are:

1. To create new service records by entering the vehicle and service details.
2. To display all saved service records in an organised list.
3. To view the complete details of an individual service record.

4. To update the details of an existing service record.
5. To delete service records that are no longer needed, with a confirmation step.
6. To validate user input so that incomplete or invalid data is not saved.
7. To store all records locally using SQLite so that they remain available after the application is closed.

1.4 Scope of the Project

Vehicle Service Tracker is designed for individual vehicle owners who want to keep a simple record of the maintenance carried out on one or more vehicles. It can be used for cars, motorcycles, scooters, and other vehicles.

The application supports only the basic CRUD operations: adding a service record, viewing the list of records and the details of a record, editing a record, and deleting a record. Records are stored in a local SQLite database on the device.

The scope is limited to these operations. The application does not provide user login, cloud synchronisation, service reminders or notifications, reports, charts, or export of data. The data is available only on the device where it was entered, and it may be lost if the application data is cleared or the application is uninstalled. The accuracy of the stored information depends on the details entered by the user.

CHAPTER 2

SYSTEM ANALYSIS

2.1 Existing System

In the conventional approach, vehicle service details are recorded in paper service books, workshop bills, notebooks, or messages received from the service centre. Some owners also rely on memory to recall the date and nature of the last service. These methods are simple but have several drawbacks. Paper documents can be lost or damaged, handwritten entries may be incomplete, and finding a particular service entry among many bills takes time.

General-purpose tools such as notes applications or spreadsheets can also be used, but they do not provide a ready-made structure for service data. The user must decide the format, enter every detail manually each time, and manage corrections without any validation. Large vehicle-management applications are available, but they often include many features, require accounts or internet access, and may be more complex than a typical owner needs.

These approaches make it inconvenient to maintain a complete and accurate service history. A simple, focused application is therefore useful for storing and managing this information.

2.2 Proposed System

Vehicle Service Tracker provides a lightweight Android application dedicated to recording vehicle service details. It offers a structured form for entering information, a list view for browsing all records, a details screen for reading a complete record, and options to edit or delete a record. All data is stored locally in an SQLite database, so the application works offline and does not depend on any external service.

The proposed system keeps the functionality deliberately limited to Create, Read, Update, and Delete operations. This keeps the interface simple and easy to learn. Input validation helps maintain the quality of the stored data, and a confirmation dialog prevents records from being deleted by mistake. The practical usefulness of the application depends on the user entering service details accurately and regularly.

CHAPTER 3

SYSTEM DESIGN AND IMPLEMENTATION

3.1 System Requirements

The system requirements of Vehicle Service Tracker describe the hardware and software needed to develop, build, and run the application. The requirements include the development environment, the Android execution environment, and the local database support.

3.1.1 Hardware Requirements

Component	Basic / recommended specification	Purpose
Computer	Laptop or desktop with a 64-bit processor	Running Android Studio and developing the application
Processor	Intel Core i5 or equivalent	Building the project and running development tools
RAM	8 GB recommended	Running Android Studio, Gradle, and the emulator
Storage	At least 10 GB of free space recommended for project files, tools, and SDK components	Storing project resources and development tools
Android device	Smartphone or tablet running Android 7.0 (API 24) or later	Running and testing the application

3.1.2 Software Requirements

Software / technology	Requirement / purpose
Android Studio	Application development
Java and Android XML	Application logic and interface design
Android SDK	Minimum API 24 (Android 7.0); compile and target API 36
SQLite (SQLiteOpenHelper)	Local database for storing service records
RecyclerView	Displaying the list of service records
Material Components	User interface elements such as buttons, text fields, and dialogs

3.2 System Architecture

The system architecture of Vehicle Service Tracker illustrates the interaction between the user interface, the application logic, and the local database. As shown in Figure 3.1, the application is organised into a presentation layer, an application logic layer, a data access layer, and the SQLite database.

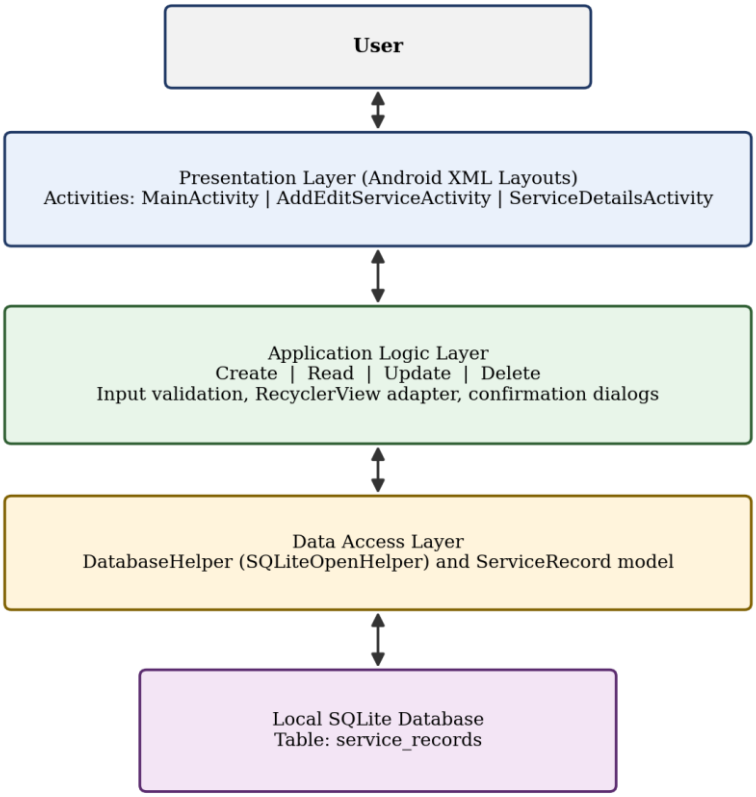


Figure 3.1 System Architecture of Vehicle Service Tracker

The presentation layer consists of the activities and XML layouts through which the user interacts with the application. The home screen shows the list of saved service records, the add/edit screen provides a form for entering or modifying details, and the details screen displays a single record in full.

The application logic layer handles user actions such as saving a record, opening a record, editing, and deleting. It validates the data entered by the user, prepares it for storage, and updates the list shown on the screen. A RecyclerView adapter binds the stored records to the list items.

The data access layer is implemented by a DatabaseHelper class that extends SQLiteOpenHelper. It creates the database and table, and provides methods for inserting, retrieving, updating, and deleting records. A ServiceRecord model class carries the data of one record between the screens and the database.

This architecture keeps the interface, logic, and data handling separate, which makes the application easier to understand and maintain. Since all data is stored in a local database, the application does not require an internet connection.

3.3 Functional Modules and Workflow

Vehicle Service Tracker is organised into functional modules corresponding to the four CRUD operations, supported by a local database module. The modules and the screens that implement them are summarised below.

Operation	Screen	Description
Create	Add Service screen	Accepts service details, validates them, and inserts a new record
Read	Home screen and Details screen	Lists all records and shows the full details of a selected record
Update	Edit Service screen	Loads an existing record, allows changes, and saves the updated values
Delete	Details screen (confirmation dialog)	Removes the selected record after the user confirms

3.3.1 Create Module

This module allows the user to add a new service record. The user enters the vehicle name, registration number, service type, service date, odometer reading, cost, next due date, and optional notes. Dates are selected using a date picker. When the user taps the save button, the entered data is validated and, if valid, stored in the database.

3.3.2 Read Module

This module displays the stored information. The home screen lists all service records, showing key details such as the vehicle name, service type, and service date. When the user selects a record, the details screen shows all stored fields of that record. If no records exist, the home screen shows a message indicating that no service records have been added.

3.3.3 Update Module

This module allows the user to correct or modify an existing record. The edit screen opens with the current values filled in. After the user changes the required fields and saves, the same record is updated in the database without creating a duplicate.

3.3.4 Delete Module

This module removes a record that is no longer needed. When the user chooses to delete a record, a confirmation dialog is displayed. The record is deleted from the database only if the user confirms; otherwise the record remains unchanged.

3.3.5 Local Database Module

This module manages the SQLite database. It creates the service_records table when the application is first run and provides the methods used by the other modules to insert, query, update, and delete records. All records are stored on the device and remain available after the application is closed.

3.3.6 Application Workflow

The overall workflow of Vehicle Service Tracker is shown in Figure 3.2. The user opens the application and sees the list of service records, then selects an action: adding a record, opening a record, editing a record, or deleting a record. Each action performs the corresponding database operation, after which the updated list is shown on the home screen.

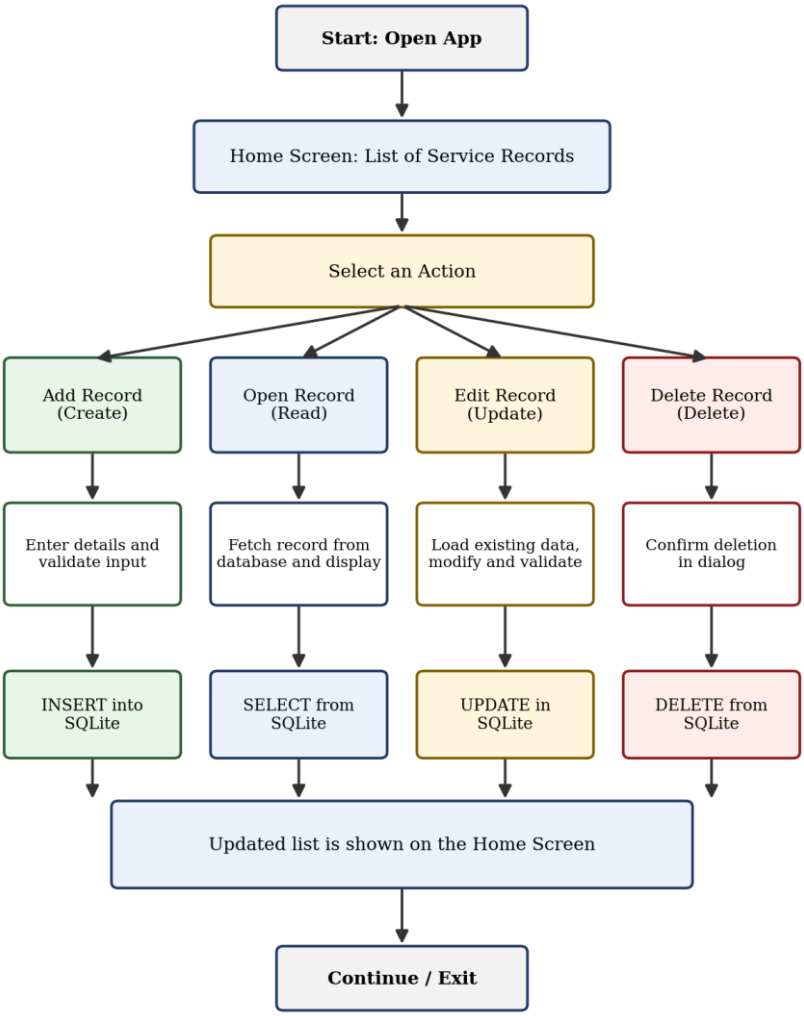


Figure 3.2 Workflow of Vehicle Service Tracker

3.4 Implementation Details

Vehicle Service Tracker is implemented as a Java-based Android application using XML layouts for its user interface. The application consists of three activities, a RecyclerView

adapter, a model class, and a database helper class that together provide the CRUD functionality.

3.4.1 Database Design

The application uses a single SQLite table named service_records, created by the DatabaseHelper class in its onCreate() method. Each row represents one service record. The structure of the table is given below.

Column	Data type	Description
id	INTEGER (PK)	Unique identifier, generated automatically for every record
vehicle_name	TEXT	Name or model of the vehicle (required)
vehicle_number	TEXT	Registration number of the vehicle (required)
service_type	TEXT	Type of service carried out, such as oil change or general service (required)
service_date	TEXT	Date of service, stored in yyyy-MM-dd format (required)
odometer_reading	INTEGER	Odometer reading in kilometres at the time of service
cost	REAL	Amount spent on the service
next_due_date	TEXT	Date when the next service is due (optional, stored for reference only)
notes	TEXT	Additional remarks (optional)

The date is stored in yyyy-MM-dd format so that records can be sorted correctly, and it is shown to the user in dd/MM/yyyy format. The DatabaseHelper class also contains the methods addService(), getAllServices(), getServiceById(), updateService(), and deleteService(), which are used by the activities.

3.4.2 Adding Service Records

The AddEditServiceActivity displays the form for entering service details. A DatePickerDialog is used for the date fields so that the user does not have to type dates manually. When the save button is pressed, the activity checks that the required fields are filled and that the odometer reading and cost contain valid non-negative numbers. If validation fails, an error message is shown on the relevant field and nothing is saved.

If the data is valid, a ServiceRecord object is created and passed to the addService() method. The method places the values in a ContentValues object and inserts them into the table using

the SQLite insert() method. After a successful insertion, a confirmation message is displayed and the user is returned to the home screen.

3.4.3 Viewing Service Records

The MainActivity retrieves all records by calling getAllServices(), which queries the table and returns the rows ordered by service date, latest first. The records are shown in a RecyclerView using the ServiceAdapter class, with each list item displaying the vehicle name, service type, and date. A floating action button on this screen opens the form for adding a new record.

When an item is tapped, the ServiceDetailsActivity is opened with the record's identifier. This activity retrieves the record through getServiceById() and displays all its fields. The list is refreshed whenever the home screen is displayed again, so that changes are visible immediately.

3.4.4 Updating Service Records

From the details screen, the user can choose the edit option. The AddEditServiceActivity is reused for this purpose: the record's identifier is passed to it, the existing values are loaded into the form fields, and the screen title changes to indicate editing. After the user modifies the data and saves, the same validation is applied and the updateService() method is called. This method uses the SQLite update() method with the record's id in the where clause so that only the selected record is changed.

3.4.5 Deleting Service Records

The delete option on the details screen displays an AlertDialog asking the user to confirm the deletion. If the user confirms, the deleteService() method is called, which uses the SQLite delete() method with the record's id. The user is then returned to the home screen, where the record no longer appears. If the user cancels, the dialog is dismissed and no data is changed.

3.4.6 Implementation Limitations

The application is limited to the CRUD operations on a single table. It does not provide search, filtering, reminders, backup, or export of data. Records are stored only on the device, so they are removed if the application data is cleared or the application is uninstalled. The next due date is stored and displayed but does not trigger any notification. Validation checks the format and presence of the data, but it cannot verify that the details entered by the user are factually correct.

CHAPTER 4

RESULTS AND DISCUSSION

4.1 Application Testing and Results

Vehicle Service Tracker was tested to verify its core functions, namely creating, reading, updating, and deleting service records, along with input validation and data persistence. The results discussed in this chapter are based on the functionality that was tested.

4.1.1 Create Operation

The create operation was tested by entering the details of a service and saving it. The application accepted the valid data, stored the record, and displayed it in the list on the home screen. When required fields were left empty, the application showed an error message and did not save the record.

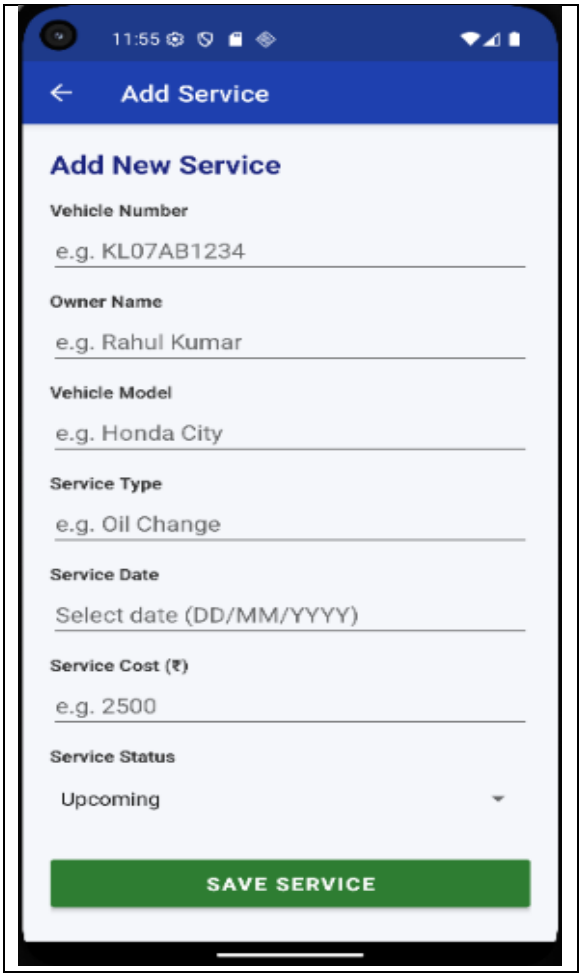


Figure 4.1 Add Service Screen

4.1.2 Read Operation

The read operation was tested by viewing the list of saved records and opening individual records. The home screen displayed all saved records, and selecting a record opened the

details screen showing all stored fields correctly. When the database contained no records, the home screen showed a message indicating that no records were available.

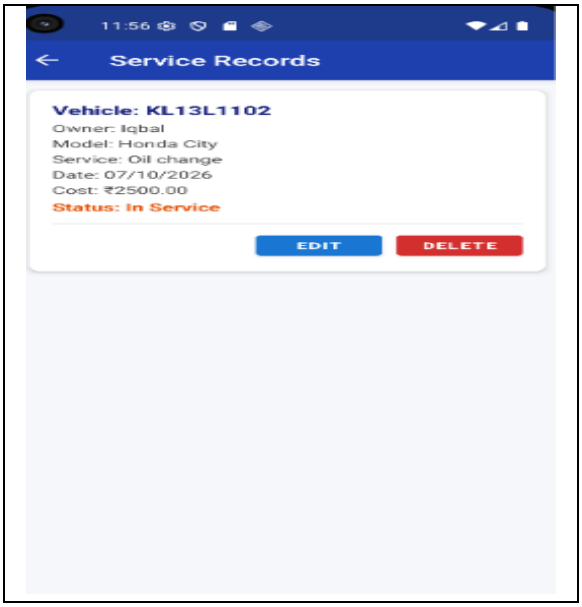


Figure 4.3 Service Details Screen

4.1.3 Update Operation

The update operation was tested by opening an existing record in the edit screen, changing one or more fields, and saving. The form was filled with the existing values, and the changes were reflected in the details screen and in the list after saving. No duplicate record was created.

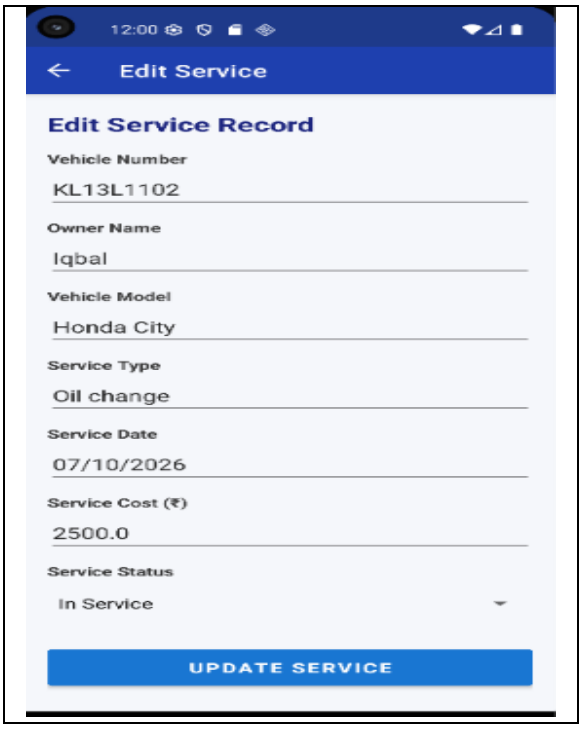


Figure 4.4 Edit Service Screen

4.1.4 Delete Operation

The delete operation was tested by selecting the delete option for a record. A confirmation dialog appeared before deletion. After confirmation, the record was removed from the database and no longer appeared in the list. When the deletion was cancelled, the record remained unchanged.

4.1.5 Functional Testing Summary

The main functions of Vehicle Service Tracker were tested to verify that the application performed the intended operations. The following table summarises the results.

Test case	Expected result	Observed result	Status
Add record with valid data	Record is saved and appears in the list	Record saved and displayed in the list	Pass
Add record with empty required fields	Error is shown and record is not saved	Error message shown; nothing saved	Pass
Invalid numeric input	Negative or non-numeric values are rejected	Invalid values not accepted	Pass
View list of records	All saved records are displayed	All records displayed	Pass
View record details	Complete details of the selected record are shown	Details displayed correctly	Pass
Empty list	A message is shown when no records exist	Empty-list message displayed	Pass
Update record	Changes are saved to the same record	Record updated; no duplicate created	Pass
Delete record (confirm)	Record is removed after confirmation	Record deleted	Pass
Delete record (cancel)	Record is retained	Record unchanged	Pass
Data persistence	Records remain after restarting the app	Records available after restart	Pass

4.2 Results and Discussion

The testing results show that Vehicle Service Tracker performs its main functions correctly. Service records could be created, listed, viewed, edited, and deleted, and the changes were reflected on the screen immediately. Validation prevented incomplete or invalid data from being saved, and the confirmation dialog protected against accidental deletion.

Because the application uses a local SQLite database, the saved records remained available after the application was closed and reopened, and the application worked without an internet connection. The use of a single reusable form for adding and editing records kept the interface simple and reduced duplicated code.

However, the application is limited to basic CRUD operations. It does not provide search, reminders, backup, or data export, and the data is stored only on the device. Testing was limited to functional checks of the implemented features; performance testing with a very large number of records was not carried out.

Overall, the verified results indicate that the application achieved its primary functional objectives under the tested conditions.

CHAPTER 5

CONCLUSION AND FUTURE WORK

5.1 Conclusion

Vehicle Service Tracker was developed as an Android application to simplify the recording and management of vehicle service history. It provides the four basic data operations (Create, Read, Update, and Delete) through a simple interface and stores all information locally using SQLite.

The application allows users to add service records, view them in a list, open complete details, edit stored information, and delete records after confirmation. Input validation helps keep the stored data consistent, and local storage allows the application to work offline.

Testing confirmed that the application performed its core functions under the conditions evaluated. The project helped in applying important mobile application development concepts, including activities, XML layouts, RecyclerView, dialogs, input validation, and SQLite database handling.

Overall, Vehicle Service Tracker demonstrates how a simple CRUD-based mobile application can replace paper records and make vehicle service information easier to maintain and review.

5.2 Future Work

The application can be improved in several ways:

- Service reminders: Notify the user when the next service date or odometer reading is approaching.
- Search and filtering: Allow records to be searched or filtered by vehicle, service type, or date.
- Multiple vehicles: Maintain separate vehicle profiles and group service records under each vehicle.
- Backup and export: Provide cloud backup or export of records to PDF or spreadsheet files.
- Expense reports: Display summaries and charts of service costs over time.

These improvements could increase the usefulness of the application and make it suitable for wider everyday use by vehicle owners.